

EasyFuel — Ozow Payment Integration

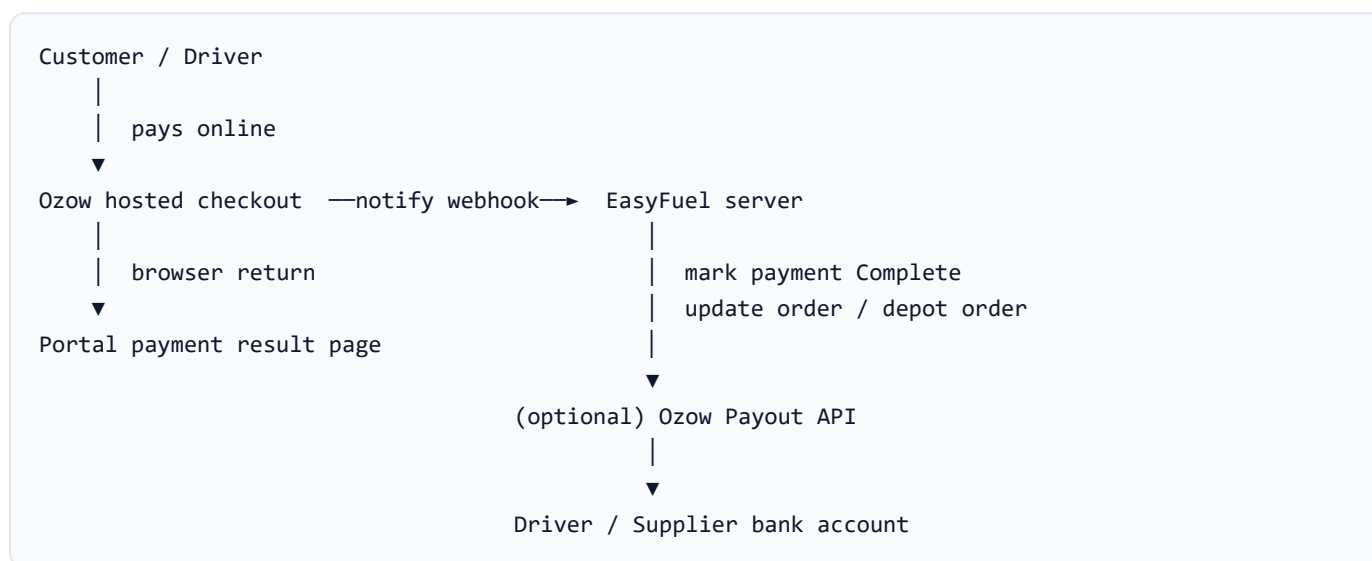
How payments work: overall flow, pay-in, payout, keys, and environments

Portal: <https://portal.easyfuel.ai> · Provider: Ozow (South Africa) · Internal reference guide

1. Payment flow (big picture)

EasyFuel uses **Ozow** for two different money movements. They look related, but they use **different APIs and different keys**.

1. **Pay-in** — customer or driver pays EasyFuel (order payment or depot order payment).
2. **Payout** — EasyFuel sends money to a driver or supplier bank account after work is complete.



Step A — Create payment

User starts checkout in web or mobile. Server creates a `payment_transactions` row, then asks Ozow for a checkout URL.

Step B — User pays on Ozow

User is redirected to Ozow's hosted page and completes (or cancels) payment.

Step C — Ozow notifies EasyFuel

Ozow calls the **notify webhook**. This is the source of truth that marks the payment complete and updates the order.

Step D — User returns to portal

Success / Cancel / Error return URLs bring the user back to EasyFuel's payment result pages.

Step E — Payout (when applicable)

After a verified pay-in, the system may submit a payout to the payee's bank details via Ozow Payouts API.

Important: Browser return pages show the user a result. The **notify webhook** is what actually confirms money and updates business state. Both must be configured correctly.

2. Pay-in (money in)

Pay-in uses Ozow **One API**. The server obtains an OAuth token, creates a payment, and returns a hosted checkout URL for web or mobile.

2.1 What pay-in is used for

- Customer delivery order payments (order awaiting payment).
- Driver → depot order online payments (depot order pending payment).

2.2 How pay-in works

1. Server checks Ozow is configured (`site code` , client credentials, `PUBLIC_APP_URL`).
2. If `OZOW_PAYIN_DRY_RUN=true` , no Ozow call is made — user is sent straight to a local success page (dev only).
3. Otherwise: get One API access token → `POST /v1/payments` → receive checkout URL.
4. User pays on Ozow.
5. Ozow hits `/api/webhooks/ozow-payin` (signed with merchant private key).
6. Server maps Ozow status (e.g. Complete) and updates the linked order / depot order.
7. User lands on `/payment/success` , `/payment/cancel` , or `/payment/error` via short return routes.

2.3 Return & notify URLs

Built from `PUBLIC_APP_URL` (must be HTTPS). Also register the same URLs in Ozow Merchant Admin → Site → URLs.

Purpose	URL
Success (browser)	<code>https://portal.easyfuel.ai/api/ozow/return/success</code>
Cancel (browser)	<code>https://portal.easyfuel.ai/api/ozow/return/cancel</code>
Error (browser)	<code>https://portal.easyfuel.ai/api/ozow/return/error</code>
Notify (server webhook)	<code>https://portal.easyfuel.ai/api/webhooks/ozow-payin</code>

If Success/Cancel/Error are missing in Ozow admin (especially on staging with `OZOW_IS_TEST=true`), the browser may stop on Ozow's `responsetest.php` page and never return to the portal.

2.4 Pay-in keys & settings

Variable	Role
<code>PUBLIC_APP_URL</code>	Base URL for return + notify links. Production: <code>https://portal.easyfuel.ai</code> . Forced to HTTPS.
<code>OZOW_SITE_CODE</code>	Merchant / site identifier in Ozow.
<code>OZOW_CLIENT_ID</code>	One API OAuth client ID (create checkout sessions).
<code>OZOW_CLIENT_SECRET</code>	One API OAuth client secret. Used for token request (and legacy hash fallback).
<code>OZOW_PRIVATE_KEY</code>	Merchant private key from Ozow dashboard. Verifies pay-in webhook Hash (SHA512).
<code>OZOW_ONE_API_BASE_URL</code>	API host. Staging: <code>https://stagingone.ozow.com</code> . Live: <code>https://one.ozow.com</code> .
<code>OZOW_IS_TEST</code>	<code>true</code> = test payment flag sent to Ozow. <code>false</code> for real production payments.
<code>OZOW_PAYIN_DRY_RUN</code>	<code>true</code> = skip Ozow entirely (fake success URL). Local / demo only.
<code>OZOW_WEBHOOK_SKIP_VERIFY</code>	Only with <code>OZOW_IS_TEST</code> : allow mismatched webhook hash. Never use in production.

Pay-in is considered **configured** when site code, client ID, client secret, and `PUBLIC_APP_URL` are all set.

3. Payout (money out)

Payout uses Ozow **Payouts API** — a separate product from One API pay-in. It sends net amounts to driver or supplier bank accounts after a successful / verified payment.

3.1 How payout works

1. Pay-in webhook marks the payment Complete and related business state is updated.
2. Server looks up the payee (driver or supplier) and their saved bank details.
3. If bank details are missing, payout is blocked and a warning / notification path is used.
4. Server matches the bank name to an Ozow `bankGroupId` (from “get available banks”).
5. If `OZOW_PAYOUT_DRY_RUN=true` , payout is simulated and not sent to Ozow.
6. Otherwise: submit payout request to Ozow Payouts API.
7. Ozow may call payout notification / verification webhooks on your server.

3.2 Payout webhook URLs

Purpose	Default URL
Payout notification	<code>https://portal.easyfuel.ai/api/webhooks/ozow-payout-notification</code>
Payout verification	<code>https://portal.easyfuel.ai/api/webhooks/ozow-payout-verification</code>

These can be overridden with `OZOW_PAYOUT_NOTIFICATION_URL` / `OZOW_PAYOUT_VERIFICATION_URL` if needed.

3.3 Payout keys & settings

Variable	Role
<code>OZOW_SITE_CODE</code>	Same site code as pay-in (shared merchant identity).
<code>OZOW_PAYOUT_API_KEY</code>	Dedicated Payouts API key from Ozow. Do not reuse the One API client secret.
<code>OZOW_PAYOUT_ACCESS_TOKEN</code>	Optional shared secret to authenticate payout webhook callbacks.
<code>OZOW_PAYOUT_API_BASE_URL</code>	Payouts API host. Default staging: <code>https://stagingpayoutsapi.ozow.com</code> . Set production URL for live.
<code>OZOW_PAYOUT_SUBMIT_PATH</code>	Submit path (default <code>/v1/requestpayout</code>).
<code>OZOW_PAYOUT_DRY_RUN</code>	<code>true</code> = do not call Ozow payouts (simulate only).
<code>OZOW_IS_TEST</code>	Also influences payout test behaviour where applicable.

Payout is considered **configured** when `OZOW_PAYOUT_API_KEY` and `OZOW_SITE_CODE` are set.

4. How it all works together

4.1 End-to-end example (delivery payment)

1. Customer places an order → order enters awaiting payment.
2. Customer chooses online payment → EasyFuel creates a payment transaction and Ozow checkout URL.
3. Customer pays on Ozow.
4. Ozow notify webhook → payment marked Complete → order progresses (e.g. paid / assignable).
5. Customer sees success page on the portal / app.
6. Later, when payout rules apply, EasyFuel may transfer the driver's share to their bank via Payouts API.

4.2 End-to-end example (depot order payment)

1. Driver has a depot order in `pending_payment` .
2. Driver starts online payment → same Ozow pay-in path, with context stored so returns know it is a depot order.
3. Ozow notify → depot order payment verified / updated.
4. Where configured, supplier/depot payout may follow once bank details exist.

4.3 Environments (what to turn on)

Mode	Suggested settings
Local / UI demo	<code>OZOW_PAYIN_DRY_RUN=true</code> (optional payout dry-run too). No real Ozow calls.
Staging with Ozow	Dry-run off · <code>OZOW_IS_TEST=true</code> · staging One API + staging payouts URLs · URLs registered in Ozow test site.
Production	Dry-run off · <code>OZOW_IS_TEST=false</code> · <code>OZOW_ONE_API_BASE_URL=https://one.ozow.com</code> · live payout base · real keys · webhook verify on · <code>PUBLIC_APP_URL=https://portal.easyfuel.ai</code> .

4.4 Where secrets live

- All Ozow keys and flags live on the **server** `.env` (e.g. Contabo `/opt/easyfuel/.env`).
- The mobile app only needs the API base URL (e.g. `EXP0_PUBLIC_API_BASE_URL`). It does **not** hold Ozow secrets.
- Never commit real keys to git. Keep them in server env / password manager only.

Quick mental model:

Pay-in = customer/driver → EasyFuel (One API + client ID/secret + private key).

Payout = EasyFuel → driver/supplier bank (Payouts API + payout API key).

Webhook notify = truth for "paid"; return URLs = UX for the user coming back.